

THỰC HÀNH CƠ SỞ LẬP TRÌNH 2

ĐỆ QUY

---o0o---

Yêu cầu:

- Xem tài liệu “Giải thuật và lập trình” để hiểu rõ phương pháp đệ quy.
- Trình bày ý tưởng chính để xây dựng công thức đệ quy (điểm dừng và đệ quy).
- Cho ví dụ minh họa cách chạy đệ quy: vẽ cây đệ quy và bảng chạy tay.
- Lập trình giải thuật đệ quy và ứng dụng ngăn xếp để khử đệ quy.

Bài 1. FIBONANCY

Cho công thức quy nạp tính số Fibonancy sau:
$$\begin{cases} F_1 = F_2 = 1 \\ F_n = F_{n-1} + F_{n-2} \quad (n > 2) \end{cases}$$

Yêu cầu: Nhập vào n, hãy tính F_n .

Dữ liệu vào: Nhập từ tập tin văn bản **FIBO.INP** chứa nhiều dòng, mỗi dòng 1 con số.

Kết quả: Xuất ra tập tin **FIBO.OUT** tính số Fibonancy tương ứng với dữ liệu nhập.

Ví dụ:

FIBO.INP	FIBO.OUT
3	2
4	3
5	5

Bài 2. THÁP HÀ NỘI

Cho 3 cọc A, B và C. Cho N đĩa kích thước đôi một khác nhau, được xếp chồng đĩa nhỏ nằm trên đĩa lớn bắt đầu tại cọc A. Nguyên tắc chuyển: một lần chỉ di chuyển một đĩa, một đĩa chỉ có thể được đặt lên một đĩa lớn hơn.

Yêu cầu: Hãy chỉ ra cách chuyển đĩa để cho chuyển hết đĩa từ cọc A sang cọc B lấy cọc C làm trung gian.

Dữ liệu vào: Nhập từ tập tin văn bản **THAPHN.INP**:

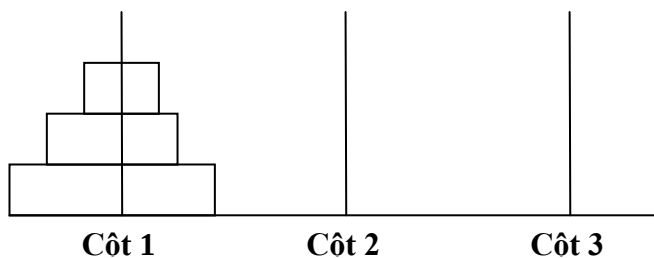
- Dòng đầu chứa N A B C với số đĩa di chuyển N, cọc ban đầu A, cọc chuyển đĩa B, cọc trung gian .

Kết quả: Xuất ra tập tin **THAPHN.OUT**:

- Dòng đầu chứa số K với K là số lần thao tác di chuyển
- K dòng tiếp, mỗi dòng chứa hai số U và V với U là cột chứa đĩa cần chuyển và V là cột chuyển đĩa tới.

Ví dụ:

THAPHN.INP	THAPHN.OUT
2 1 2 3	3
	1 3
	1 2
	3 2

**Bài 3. Mọi con đường đều về 0**

Một số tự nhiên dương N đều tìm được hai số nguyên dương $x \leq y$ sao cho $N = x.y$. Với cách phân tích này ta sẽ hình thành ra số tự nhiên mới $N1 = (x-1)(y+1)$. Số này có thể dương hoặc bằng 0. Nếu nó dương ta có thể chọn một cách phân tích nào đó và lặp lại những thao tác trên.

Yêu cầu: Với số tự nhiên N ($N \leq 10000$) cho trước, hãy cho biết ta có thể sinh ra được những số nào.

Dữ liệu vào: Nhập từ tập tin văn bản **VE0.INP** trong đó chứa một số duy nhất N.

Kết quả: Xuất ra tập tin văn bản **VE0.OUT** liệt kê tất cả các số tìm được theo thứ tự tăng dần.

Ví dụ:

VE0.INP	VE0.OUT
12	0 3 4 6 7 10

Bài 4. Tính định thức

Cho ma trận vuông cấp n .

Yêu cầu: Hãy tính định thức của ma trận vuông.

Dữ liệu vào: Nhập từ tập tin **DINHTHUC.INP**:

- Dòng đầu chứa số N , với N là cấp của ma trận vuông.
- N dòng tiếp theo, mỗi dòng chứa N số xác định ma trận vuông.

Kết quả: Xuất ra tập tin **DINHTHUC.OUT** chứa giá trị của định thức.

Ví dụ:

DINHTHUC.INP	DINHTHUC.OUT
2 1 2 3 4	-2

Hướng dẫn:

Cho ma trận vuông A cấp n sau: $|A| = \begin{vmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{vmatrix}$. Tính **detA** theo phương pháp phần phụ đại số.

Phần phụ đại số của phần tử a_{ij} (ký hiệu là C_{ij}) được tính theo công thức: $C_{ij} = (-1)^{i+j} \cdot \det M_{ij}$. Trong đó M_{ij} là một ma trận cấp $n-1$ suy ra từ ma trận A sau khi loại bỏ hàng i cột j của ma trận A .

Công thức phần phụ đại số: $\det A = a_{i1}C_{i1} + a_{i2}C_{i2} + \dots + a_{in}C_{in}$ (theo hàng i)
 hoặc $\det A = a_{1j}C_{1j} + a_{2j}C_{2j} + \dots + a_{nj}C_{nj}$ (theo cột j)

BÀI TẬP THÊM ĐỀ QUY**Bài 5. Tổ hợp**

Viết hàm đệ quy tính số tổ hợp chập k của n phần tử theo công thức sau:

$$\begin{aligned} C(n, r) &= 1 && \text{nếu } n=r \text{ hay } r=0 \\ C(n, r) &= C(n-1, r) + C(n-1, r-1) && \text{nếu } n, r \geq 0 \end{aligned}$$

Bài 6. USCLN

Viết hàm đệ quy tìm ước số chung lớn nhất của 2 số nguyên m, n :

$$\begin{aligned} \text{USCLN}(m, n) &= m && \text{nếu } n=0 \\ \text{USCLN}(m, n) &= \text{USCLN}(n, n \bmod m) && \text{nếu } n \neq 0 \end{aligned}$$

BÀI 7. TỔNG 1

Tìm công thức đệ quy và Viết hàm đệ quy tính tổng sau:

$$S = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \dots + \frac{1}{n}$$

Bài 8. TỔNG 2

Tìm công thức đệ quy và Viết hàm đệ quy tính tổng sau:

$$S = 1 - x^2 + x^3 - x^4 + \dots + (-1)^{n+1} x^n$$

Bài 9. Tìm max của dãy số

Tìm công thức đệ quy và Viết hàm đệ quy tìm số lớn nhất trong dãy số $a_1, a_2, \dots, a_n$

Bài 10. Đảo số

Tìm công thức đệ quy và Viết hàm đệ quy tìm số đảo ngược của số nguyên dương n.

Bài 11. Kiểm tra số nguyên tố

Tìm công thức đệ quy và Viết hàm đệ quy kiểm tra số nguyên n có phải là số nguyên tố không.

Bài 12a. Phân số tối giản

Tìm công thức đệ quy và Viết hàm đệ quy liệt kê các phân số tối giản không giảm nằm trong $[0, 1]$ và có mẫu số nhỏ hơn hay bằng n.

Bài 12b. Số La mã

Tìm công thức đệ quy và Viết hàm đệ quy tính giá trị của một số được viết dưới dạng chữ số LA MÃ.

Ví dụ :

V $\rightarrow$ 5

VI $\rightarrow$ 6

IV $\rightarrow$ 4

---o0o---
(Hết)

HƯỚNG DẪN CƠ SỞ LẬP TRÌNH 2 ĐỆ QUY VÀ KHỬ ĐỆ QUY

---o0o---

Ý TƯỞNG ĐỆ QUY

Bài toán giải bằng đệ quy cần phải xây dựng:

+ **Điều kiện dừng**: bài toán đơn giản có thể giải quyết được.

+ **Đệ quy**: chia bài toán cha thành nhiều bài toán con và gọi lặp lại để thực hiện giải bài toán con.

Trong đó, điều kiện dừng là quan trọng nhất bởi nó quyết định tính dừng của chương trình.

Cần đọc thêm trong tài liệu tham khảo

Bài 1. FIBONANCY

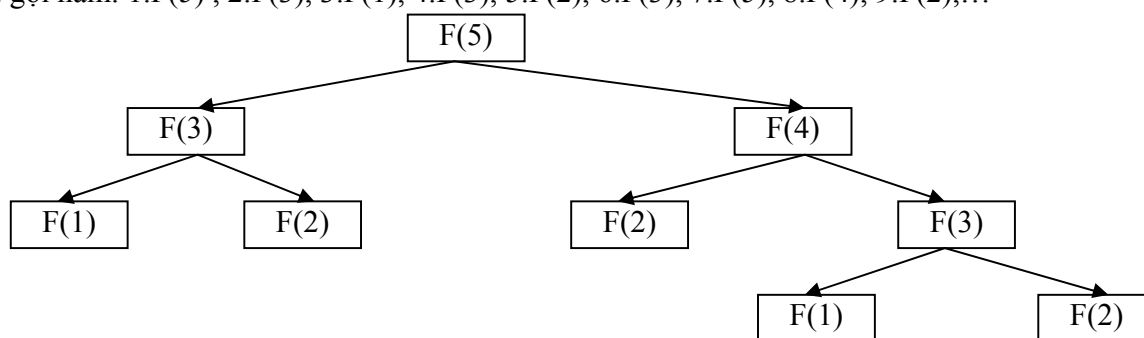
Công thức đệ quy:
$$\begin{cases} F_1 = F_2 = 1 \\ F_n = F_{n-1} + F_{n-2} \quad (n > 2) \end{cases}$$

Hàm đệ quy như sau:

```
int Fibo_DQ(int n)
{
    // dieu kien dung
    if(n<=2)
    {
        return 1;
    }
    else
    {
        // buoc de quy
        return Fibo_DQ(n-1) + Fibo_DQ(n-2);
    }
} // Fibo_DQ
```

Cây đệ quy: F(5) gọi → F(3) gọi → F(1) quay về → F(3) gọi → F(2) quay về → F(3) quay về → F(5) gọi → F(4) gọi → F(2) quay về → ...

Thứ tự gọi hàm: 1.F(5) , 2.F(3), 3.F(1), 4.F(3), 5.F(2), 6.F(3), 7.F(5), 8.F(4), 9.F(2),...



Lập bảng chạy tay:

STT	N	Fibo	Return	Giải thích
1	5	F_5	$F_3 + F_4$	---
2	3	$F_3 F_3$	$F_1 + F_2$	---
3	1	$F_3 F_3 F_1$	1	điều kiện dừng
4	3	$F_3 F_3$	$1 + F_2$	Quay lại F_3
5	2	$F_3 F_3 F_2$	1	điều kiện dừng
6	3	$F_3 F_3$	$1 + 1 \rightarrow 2$	hoàn thành
...	...	...	...	...

Ý TƯỞNG KHỬ ĐỆ QUY

Đệ quy cũng chính là một thể hiện của cơ chế LIFO (Last In First Out), hàm cha thực hiện trước gọi hàm con, và hàm con sau khi hoàn thành quay lại hàm cha. Trước khi chương trình thực hiện hàm con, chương trình lưu các biến cục bộ và biến tham trị trong hàm cha. Sau khi hoàn thành hàm con, chương trình sẽ phục hồi dữ liệu.

Để khử đệ quy, ta dùng cấu trúc dữ liệu Stack thể hiện cơ chế LIFO. Ta sẽ đưa trạng thái ban đầu (dữ liệu của hàm cha cần lưu) vào Stack. Tại mỗi bước, thực hiện thao tác lấy Stack. Nếu rơi vào nút lá hay điều kiện dừng thì giải quyết. Nếu là nút trung gian hay bước đệ quy thì sinh các bài toán con bỏ vào Stack.

Mã giả cài đặt:

* s_0 : trạng thái ban đầu, s_i : trạng thái bài toán ở bước i .

* stack có các thao tác: push, pop, top, empty

```
push(stack, s0)
while (stack != rỗng)
{
    popStack(stack, si);
    if (si là nút lá)
        giải bài toán si
    else if (si là nút trung gian)
        sinh bài toán con
} // while
```

Lập bảng chạy tay với $N = 5$

STT	S_i	Stack	Tong	Giải thích
0	0	5	0	Khởi tạo
1	5	43	0	Nút trung gian
2	3	421	0	Nút trung gian
3	1	42	1	Nút lá (cộng 1 vào tổng)
4	2	4	2	Nút lá (cộng 1 vào tổng)
5	4	32	2	Nút trung gian
...	...	...	...	...

```
int Fibo_Stack(int n)
{
    TStack t; int tong, si;
    initStack(t); tong = 0;
    pushStack(t, n);
    while (!emptyStack(t))
    {
        popStack(t, si);
        if (si <= 2) tong = tong + 1;
        else
        {
            pushStack(t, si-1);
            pushStack(t, si-2);
        }
    } // while
} // Fibo_Stack
```

XÂY DỰNG CẤU TRÚC STACK

Lưu ý:

- Đối với một dự án, ta có thể có nhiều tập tin cpp và h nhưng chỉ có một tập tin cpp chứa hàm main.
- Xây dựng một thư viện:
 - o Hai tập tin: stack.cpp và stack.h
 - o Stack.h chứa các khai báo về dữ liệu và tên hàm công khai (cho người dùng). Để tránh include hai lần cần để trong macro sau:

```
#ifndef _STACK_H_
#define _STACK_H_
Khai báo dữ liệu và hàm công khai
#endif // _STACK_H_
```

- o Stack.cpp chứa cài đặt hàm công khai và hàm riêng cùng dữ liệu cục bộ.
- o Namespace dùng để tránh trùng tên.

Stack.h

```
#ifndef _STACK_H_
#define _STACK_H_

#define Data int
#define MAX 1000

namespace CSLT2
{
    namespace Stack
    {

        struct TStack
        {
            int top;
            Data aData[MAX];
        };

        void initStack(TStack &s);

        int pushStack(TStack &s, Data x);

        int popStack(TStack &s, Data &x);

        int topStack(TStack &s, Data &x);

        int emptyStack(TStack &s);

        int fullStack(TStack &s);

    } // Stack (namespace)
} // CSLT2 (namespace)

#endif // _STACK_H_
```

Stack.cpp

```
#include <cstdio>
#include <cstring>
#include <iostream>
#include "Stack.h"

using namespace std;

namespace CSLT2
```

```
{
    namespace Stack
    {
void initStack(TStack &s)
{
    s.top = -1;
    memset(s.aData,0,sizeof(s.aData));
} // initStack

int pushStack(TStack &s,Data x)
{
    if(fullStack(s)) return 0;

    s.top++;
    s.aData[s.top] = x;

    return 1;
} // pushStack

int popStack(TStack &s,Data &x)
{
    if(emptyStack(s)) return 0;

    x = s.aData[s.top];
    s.top--;

    return 1;
} // popStack

int topStack(TStack &s,Data &x)
{
    if(emptyStack(s)) return 0;

    x = s.aData[s.top];

    return 1;
} // topStack

int emptyStack(TStack &s)
{
    return s.top==-1;
} // emptyStack

int fullStack(TStack &s)
{
    return s.top==MAX-1;
} // fullStack

    } // Stack (namespace)
} // CSLT2 (namespace)
```

---o0o---
(Hết)

THỰC HÀNH CƠ SỞ LẬP TRÌNH 2 TẬP CON, HOÁN VỊ VÀ ỨNG DỤNG

---o0o---

Yêu cầu

- Tìm hiểu phương pháp vét cạn (brute-force), quay lui (back-tracking).
- Ứng dụng giải quyết bài toán tổ hợp tập con và tổ hợp dùng đệ quy: nêu ý tưởng, vẽ cây đệ quy và lập bảng chạy tay.
- Tìm hiểu phương pháp cài đặt khử đệ quy, không đệ quy cho bài toán tổ hợp, tập con.

Bài 1. LIỆT KÊ XÂU NHỊ PHÂN ĐỘ DÀI N

Hãy liệt kê tất cả các chuỗi nhị phân $x_1x_2\dots x_n$ với $x_i \in \{0,1\}$, theo thứ tự từ điển.

Dữ liệu: Nhập từ tập tin **XAUNP.INP**, chứa một số nguyên dương N.

Kết quả: Xuất ra tập tin văn bản **XAUNP.OUT**, gồm:

- Dòng đầu chứa K với K là số lượng chuỗi nhị phân liệt kê.
- K dòng tiếp, mỗi dòng chứa một chuỗi nhị phân. Biết các chuỗi nhị phân được liệt kê theo thứ tự từ điển.

Ví dụ:

XAUNP.INP	XAUNP.INP
2	4
	00
	01
	10
	11

Câu hỏi:

1. Phương pháp quay lui:
 - a. Nêu ý tưởng tổng quát phương pháp quay lui (đệ qui, không đệ qui).
 - b. Nêu thuật toán quay lui cho bài toán liệt kê tập con.
 - c. Lập bảng chạy tay và vẽ cây đệ quy để minh họa thuật toán.
2. Lập trình giải quyết bằng ngôn ngữ C/C++ (đệ qui - khử đệ qui, không đệ qui).

Bài 2. LIỆT KÊ HOÁN VỊ N SỐ NGUYÊN ĐẦU TIÊN

Hãy liệt kê tất cả hoán vị của N số nguyên đầu tiên theo thứ tự từ điển.

Dữ liệu: Nhập từ tập tin **HOANVI1.INP**, chứa một số nguyên dương N.

Kết quả: Xuất ra tập tin văn bản **HOANVI1.OUT**, gồm:

- Dòng đầu chứa K với K là số lượng hoán vị của N số nguyên đầu tiên.
- K dòng tiếp, mỗi dòng chứa một dãy hoán vị. Biết các dãy hoán vị được liệt kê theo thứ tự từ điển.

Ví dụ:

HOANVI1.INP	HOANVI1.INP
2	2
	1 2
	2 1

Bài 3. LIỆT KÊ TẬP CON

Cho tập hợp A có N phần tử đôi một khác nhau. Mỗi phần tử là một ký tự từ $A \rightarrow Z$.

Yêu cầu: Hãy liệt kê tập con của tập hợp đó.

Dữ liệu: Nhập từ tập tin **TAPCON.INP** gồm:

- Dòng đầu chứa N với N là số lượng phần tử của tập A.
- N dòng tiếp theo, mỗi dòng chứa 1 ký tự từ $A \rightarrow Z$ (các ký tự đôi một khác nhau).

Kết quả: Xuất ra tập tin văn bản **TAPCON.OUT** gồm:

- Dòng đầu ghi số nguyên K là số lượng các tập con.
- K dòng tiếp theo, mỗi dòng chứa một tập con của A. Với tập rỗng, xuất ra 0.

Ví dụ:

TAPCON.INP	TAPCON.OUT
2	4
F	0
B	F
	B
	F B

Bài 4. LIỆT KÊ TẬP HOÁN VỊ

Cho tập hợp A có N phần tử đôi một khác nhau. Mỗi phần tử là một ký tự từ $A \rightarrow Z$.

Yêu cầu: Hãy liệt kê các hoán vị của tập hợp đó.

Dữ liệu: Nhập từ tập tin văn bản **HOANVI2.INP** gồm:

- Dòng đầu chứa N với N là số lượng phần tử của tập A.
- N dòng tiếp theo, mỗi dòng chứa 1 ký tự từ $A \rightarrow Z$ (các ký tự đôi một khác nhau).

Kết quả: Xuất ra tập tin văn bản **HOANVI2.OUT** gồm:

- Dòng đầu chứa K với K là số lượng hoán vị.
- K dòng tiếp theo, mỗi dòng chứa một hoán vị của A.

Ví dụ:

HOANVI2.INP	HOANVI2.OUT
3	4
F	F B A
B	F A B
A	B F A
	B A F
	A B F
	A F B

Bài 5. DẪY CON

Cho dãy số nguyên $A_1, A_2, \dots, A_n$ và số nguyên M. Hãy tìm một dãy con của dãy trên sao cho:

$$A_{i1} + A_{i2} + \dots + A_{ik} = M$$

Dữ liệu: Vào từ tập tin văn bản **DAYCON.INP** gồm:

- Dòng đầu chứa số N và M với N là số lượng phần tử của dãy A, M là giá trị tổng.
- Dòng 2 chứa N số nguyên cách nhau bởi khoảng trắng là giá trị của các phần tử dãy A.

Kết quả: Xuất ra tập tin văn bản **DAYCON.OUT** gồm:

- Dòng đầu chứa K với K là số phần tử của dãy rút ra từ A có tổng bằng M.
- Dòng 2 liệt kê K số nguyên rút ra từ dãy A có tổng bằng M.

Ví dụ:

DAYCON.INP
3 5
2
3
4

DAYCON.OUT
2
2 3

Bài 6. BÀI TOÁN CẢI TÚI 0/1

Cho một va li có thể chứa W đơn vị trọng lượng. Có N đồ vật, đồ vật i có trọng lượng A_i và có giá trị C_i . Hãy xếp đồ vật vào vali để tổng giá trị của vali là lớn nhất.

Dữ liệu: Nhập từ tập tin văn bản **KNAPSACK1.INP** gồm:

- Dòng đầu chứa số nguyên dương N và W .
- N dòng tiếp, dòng i ghi hai số nguyên dương A_i và C_i (với $i=1,2,\dots,n$), cách nhau bởi dấu cách.

Kết quả: Xuất ra tập tin **KNAPSACK1.OUT** gồm:

- Dòng đầu chứa V với V là tổng giá trị lớn nhất của các đồ vật được chọn.
- Các dòng tiếp theo, mỗi dòng chứa chỉ số của đồ vật được chọn.

Ví dụ:

KNAPSACK1.INP	KNAPSACK1.OUT
3 10	600
1 70	2
5 300	3
5 300	

---o0o---

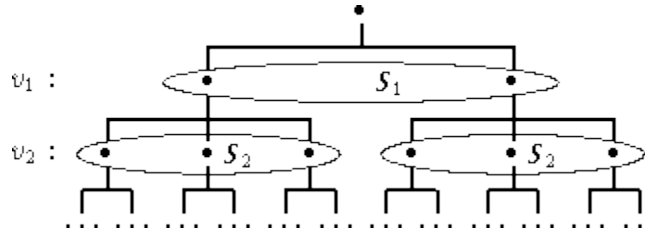
(Hết)

HƯỚNG DẪN THỰC HÀNH CƠ SỞ LẬP TRÌNH 2

TẬP CON, HOÁN VỊ VÀ ỨNG DỤNG

---o0o---

PHƯƠNG PHÁP QUAY LUI



- **Giải quyết một bài toán:** tìm một tập dữ liệu D là lời giải của bài toán khi cho trước một dãy dữ liệu biểu diễn không gian bài toán $v = (v[1], v[2], \dots, v[n])$ thỏa mãn N điều kiện và mỗi $v[i]$ nằm trong miền trị S_i .
- Chia N điều kiện thành hai phần P và Q : $N = P + Q$. P là tập điều kiện dùng để sinh các trạng thái bài toán v_i . Q là tập điều kiện kiểm tra v_i có thỏa. Nếu v_i thỏa thì đó chính là lời giải của bài toán.
- Ý tưởng phương pháp quay lui:

Bước 1. (Khởi tạo) Xuất phát từ một dãy ban đầu $v = (v[1], v[2], \dots, v[i])$ thỏa điều kiện P .

Bước 2. Nếu v thỏa Q thì ta dùng thuật toán và thông báo là dãy v .

Bước 3. Tìm một phần tử $v[i+1]$ trong miền S_{i+1} bổ sung cho v sao cho thỏa P : $v = (v[1], v[2], \dots, v[i+1])$ dựa trên miền trị của phần tử $v[i+1]$ (Bước sinh các node trung gian).

3.1. Tìm được $v[i+1]$, quay lại bước 2

3.2. Không tìm được $v[i+1]$, tức là với mọi $v[i+1]$ có thể lấy trong D , dãy $v = (v[1], v[2], \dots, v[i+1])$ không thỏa P . Điều này có nghĩa là đi theo đường $v = (v[1], v[2], \dots, v[i])$ không dẫn tới kết quả.

Đổi hướng bằng cách loại $v[i]$ ra khỏi dãy v , giảm i một đơn vị rồi quay lại Bước 3.

- Lưu ý:
 - o Cần đánh dấu $v[i]$ là phần tử đã loại tại vị trí i để sau đó không đặt vào.
 - o Bài toán dừng lại khi không thể lùi được nữa. Bài toán vô nghiệm khi đã duyệt qua mọi khả năng (vét cạn).
- **Khung sườn không đệ quy:**

try()

1. Khởi trị v sao cho v thỏa P

2. **repeat**

3. **if** v thỏa Q **then**

a. Ghi nhận nghiệm và thoát

4. **if** hết khả năng duyệt **then**

a. Ghi nhận vô nghiệm và thoát

5. **if** Tìm được một nước đi ($v[i+1]$ trong S_{i+1}) **then**

a. Tiến

6. **else**

a. Lùi

7. **until** false;

end

- **Khung sườn đệ quy:**

try ($v = (v[1], v[2], \dots, v[i])$)

```

1. if v thỏa Q then
2.     Ghi nhận nghiệm và thoát
3. for mọi v[i+1] thỏa Si+1 do
4.     if v = (v[1],v[2],...,v[i],v[i+1]) thỏa P then
5.         Ghi nhận trước khi tiến vào (nếu cần thiết)
6.         Try (v = (v[1],v[2],...,v[i],v[i+1]))
7.         Phục hồi lại trạng thái cũ (nếu cần thiết)
end

```

Bài 1. Liệt kê xâu nhị phân độ dài N

Khung chương trình cài đặt

```

#include <iostream>
#include <fstream>

using namespace std;

#define FI "XAUNP.INP"
#define FO "XAUNP.OUT"

/* Mở luồng nhập / xuất */
ifstream finp(FI);
ofstream fout(FO);

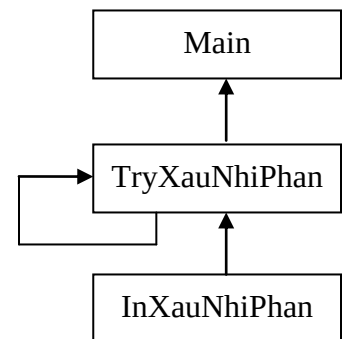
/* Biểu diễn cấu trúc dữ liệu */
#define MAX 100

int aNhiPhan[MAX]; // biểu diễn xâu nhị phân
int nNhiPhan;      // số lượng phần tử xâu nhị phân

/* Nguyên mẫu hàm */
/*
In xâu nhị phân ra màn hình
Input : aNhiPhan[] = { 0, 0, 1 }, nNhiPhan = 3
Output: luồng xuất có giá trị "0 0 1 xuống dòng"
*/
void InXauNhiPhan();

/*
Sinh tập hợp xâu nhị phân (đệ quy)
Input
+ aNhiPhan[], nNhiPhan: trạng thái hiện tại của xâu nhị phân
+ k: vị trí đang xét
Output : xuất ra màn hình xâu nhị phân
*/
void TryXauNhiPhan(int k);

```



```

int main(int argc, char *argv[])
{
    /* Kiểm tra luồng nhập/xuất mở thành công? */
    if(!finp.is_open() || !fout.is_open())
    {
        cout << "Luồng nhập/xuất mở không thành công?" << endl;
        return;
    } // if

    /* Xử lý luồng nhập/xuất */
    finp >> nNhiPhan;
    memset(aNhiPhan, -1, sizeof(aNhiPhan)); // điền giá trị -1 cho aNhiPhan
    TryXauNhiPhan(1);

    /* Đóng luồng nhập/xuất */
    finp.close();
    fout.close();
} // main
/* Cài đặt các hàm */

```

Giải thích

Biểu diễn cấu trúc dữ liệu tập hợp và tập con:

```

/* Biểu diễn cấu trúc dữ liệu */
#define MAX 100

int aNhiPhan[MAX]; // biểu diễn xâu nhị phân
int nNhiPhan;      // số lượng phần tử xâu nhị phân

```

Miền trị của từng phần tử xâu nhị phân: $aNhiPhan[i: 1 \dots nNhiPhan] = \begin{cases} -1 \\ 0 \\ 1 \end{cases}$

```

/*
In xâu nhị phân ra màn hình
Input : aNhiPhan[] = { 0, 0, 1 }, nNhiPhan = 3
Output: luồng xuất có giá trị "0 0 1 xuống dòng"
*/
void InXauNhiPhan();

```

Mã giả cài đặt:

```

InXauNhiPhan()
1. for i ← 1 to nNhiPhan do
2.     In aNhiPhan[i] và khoảng trắng
3. In dấu xuống dòng
end

```

```

/*
Sinh tập hợp xâu nhị phân (đệ quy)
Input
+ aNhiPhan[], nNhiPhan: trạng thái hiện tại của xâu nhị phân
+ k: vị trí đang xét
Output : xuất ra màn hình xâu nhị phân
*/
void TryXauNhiPhan(int k);

```

Liệt kê tập con với phương pháp quay lui (dùng đệ quy):

Để chọn một tập con trong không gian tập con P của tập hợp, ta chia làm n giai đoạn (trạng thái) $V = \{v_0, v_1, \dots, v_i, \dots, v_{n-1}\}$. Mỗi giai đoạn có hai cách chọn lựa là 0 và 1, tức là $v_i \in \{0, 1\}$ (miền xác định của v_i , ký hiệu S_i).

$$V = \left\{ \begin{array}{cccccc} 0/1 & 0/1 & 0/1 & \dots & 0/1 & 0/1 \\ 1 & 2 & 3 & \dots & n-1 & n \end{array} \right\}$$

V nằm trong tập P khi chọn đủ n phần tử lần lượt từ vị trí 1, 2, ..., n.

Mã giả cài đặt:

```

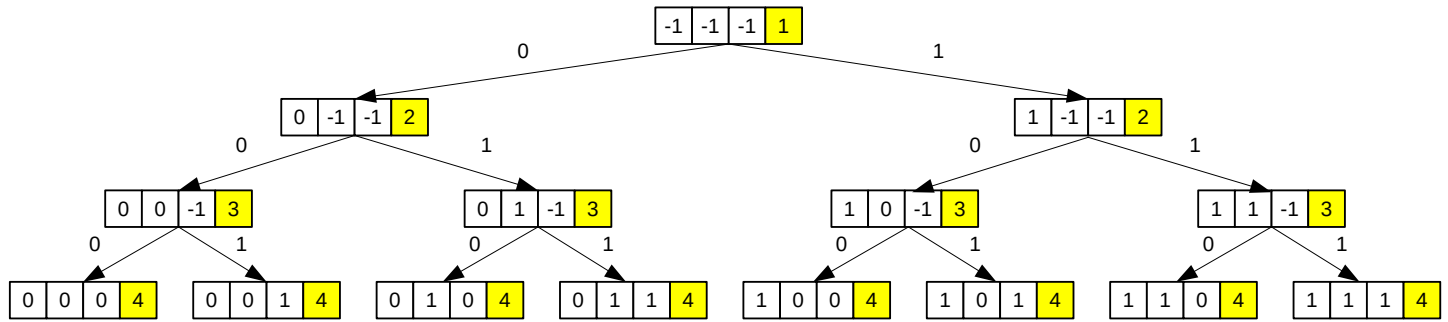
/*
Khởi tạo ban đầu:
+ aNhiPhan[] = { -1, -1, -1, ..., -1 }.
+ Gọi hàm TryXauNhiPhan(1)
*/
TryXauNhiPhan(k)
1. if k = n + 1 then
2.   InXauNhiPhan()
3.   return
4. for i ← 0 to 1 do           // chọn v_k giá trị 0 hoặc 1
5.   aNhiPhan[k] ← i          // khởi tạo trạng thái trước khi tiến vào
6.   TryXauNhiPhan(k+1);       // xét vị trí tiếp theo v_{k+1}
7.   aNhiPhan[k] ← -1         // phục hồi trạng thái
end

```

Lập bảng chạy tay, vẽ cây đệ quy, xác định thứ tự gọi hàm:

Bước	k	i	aNhiPhan[]			TryXauNhiPhan	Ghi Chú
			1	2	3		
0	1	---	-1	-1	-1	T_1	Gọi T_1
1	1	0	0	-1	-1	T_1T_2	Gán giá trị 0 vào vị trí 1, vào T_2
2	2	0	0	0	-1	$T_1T_2T_3$	Gán giá trị 0 vào vị trí 2, vào T_3
3	3	0	0	0	0	$T_1T_2T_3T_4$	Gán giá trị 0 vào vị trí 3, vào T_4
4	4	---	0	0	0	$T_1T_2T_3T_4$	In ra màn hình 0 0 0
5	3	0	0	0	-1	$T_1T_2T_3$	Quay lui về T_3 , phục hồi giá trị tại vị trí 3
6	3	1	0	0	1	$T_1T_2T_3T_4$	Gán 1 vào vị trí 3, vào T_4
7	4	---	0	0	1	$T_1T_2T_3T_4$	In ra màn hình 0 0 1
8	3	1	0	0	-1	$T_1T_2T_3$	Quay lui về T_3 , phục hồi giá trị tại vị trí 3
9	2	0	0	-1	-1	T_1T_2	Quay lui về T_2 , phục hồi giá trị tại vị trí 2
...	...	...	...	...	...	...	...

Cây đệ quy



Thứ tự gọi đệ quy: T_1 gọi T_2 gọi T_3 gọi T_4 quay về T_3 gọi T_4 ...

Hàm cài đặt không đệ quy:

```
/*
Sinh tập hợp xâu nhị phân (đệ quy)
Input
+ aNhiPhan[], nNhiPhan: trạng thái hiện tại của xâu nhị phân
Output : xuất ra màn hình xâu nhị phân
*/
void TryXauNhiPhan();
```

Mã giả cài đặt:

```
TryXauNhiPhan()
1. k ← 1; // giả sử ban đầu đặt ở vị trí 1
2. aNhiPhan[1..nNhiPhan] ← -1
3. while k > 0 do // vị trí vẫn chưa duyệt hết
4.   if k = nNhiPhan then // sinh đủ trạng thái tập con
5.     InXauNhiPhan()
6.     k-- // lùi một bước
7.   if aNhiPhan[k] < 1 then // vẫn còn tiến được
8.     aNhiPhan[k]++
9.     k++ // tiến vào trạng thái tiếp
10.  else
11.    aNhiPhan[k] ← -1 // phục hồi trạng thái
12.    k-- // lùi một bước
end
```

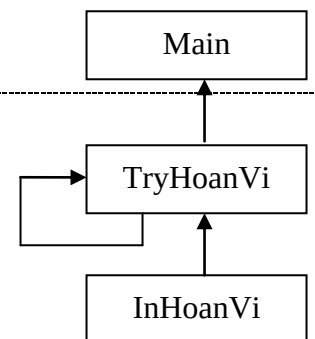
Lập bảng chạy tay chương trình quay lui không đệ quy.

Bài 2. LIỆT KÊ HOÁN VỊ N SỐ NGUYÊN ĐẦU TIÊN

Khung chương trình như bài liệt kê xâu nhị phân

Biểu diễn cấu trúc dữ liệu hoán vị:

```
#define MAX 100
int nHoanVi; // số lượng phần tử tập hoán vị
int aHoanVi[MAX]; // biểu diễn trạng thái hoán vị
```



```
bool chkDaChon[MAX]; // kiểm tra một phần tử đã được chọn vào tập hoán vị?
```

Biểu diễn hoán vị aHoanVi[] = {2,3,1,...}

Vị trí	i	1	2	3	4	...	N
Giá trị	aHoanVi[i]	2	0	3	1	...	...

Kiểm tra việc một phần tử đã có chọn vào hoán vị ? $chkDaChon[i: 1 \dots nHoanVi] = \begin{cases} true \\ false \end{cases}$

Vị trí	i	1	2	3	4	...	N
Giá trị	chkDaChon[i]	true	true	true	true	...	...

Phần tử 2, 3,1 ... đã được chọn.

```
/*
In tập hoán vị
Input: aHoanVi[] = { 2, 3, 1 }, nKyTu = 3
Output: xuất ra màn hình giá trị 2 3 1
*/
void InHoanVi();
```

```
/*
Sinh các hoán vị số nguyên (đệ quy)
Input: aHoanVi[], nHoanVi
Output: Xuất tất cả các hoanvi
*/
void TryHoanVi(int k);
```

Liệt kê hoán vị với phương pháp quay lui (dùng đệ quy):

Để chọn một hoán vị trong không gian tập con P của tập hợp, ta chia làm n giai đoạn (trạng thái) $V = \{v_0, v_1, \dots, v_i, \dots, v_{n-1}\}$. Mỗi giai đoạn có n cách chọn lựa từ 0 đến n-1, tức là $v_i \in \{0, 1, \dots, n-1\}$ sao cho **bước sau không chọn trùng bước trước** $v_i \notin \{v_0, v_1, \dots, v_{i-1}\}$ (miền xác định S_i).

$$V = \left\{ \begin{array}{cccccc} 1 \dots n & 1 \dots n & 1 \dots n & \dots & 1 \dots n & 1 \dots n \\ 1 & 2 & 3 & \dots & n-1 & n \end{array} \right\}$$

V nằm trong tập P khi chọn đủ n phần tử lần lượt từ vị trí 1, 2, ..., n.

Mã giả cài đặt:

```
/*
Khởi tạo ban đầu:
+ aHoanVi [] = { -1, -1, -1, ..., -1 }.
+ chkDaChon[] ← false
+ TryHoanVi(1)
*/
TryHoanVi(k)
1. if k = n+1 then
2.   InHoanVi()
3.   return
4. for i ← 1 to n do // chọn v_k giá trị 1 đến n sao cho không trùng bước trước
5.   if chkDaChon[i] = false then
6.     aHoanVi[k] ← i // khởi tạo trạng thái trước khi tiến vào
```



```
7.      chkDaChon[i] ← true      // đã chọn giá trị i
8.      TryHoanVi(k+1);          // xét vị trí tiếp theo  $v_{k+1}$ 
9.      aHoanVi [k] ← -1        // phục hồi trạng thái
10.     chkDaChon[i] ← false     // phục hồi chưa chọn giá trị i
end
```

```
/*
Sinh các hoán vị số nguyên (không đệ quy)
Input: aHoanVi[], nHoanVi
Output: Xuất tất cả các hoán vị
*/
void TryHoanVi();
```

---o0o---

(Hết)

THỰC HÀNH CƠ SỞ LẬP TRÌNH 2

TỔ HỢP, CHỈNH HỢP VÀ ỨNG DỤNG

---o0o---

YÊU CẦU

- Tìm hiểu phương pháp vét cạn (brute-force), quay lui (back-tracking).
- Ứng dụng giải quyết bài toán liệt kê tổ hợp và chỉnh hợp dùng đệ quy: nêu ý tưởng, vẽ cây đệ quy và lập bảng chạy tay.
- Tìm hiểu phương pháp cài đặt khử đệ quy, không đệ quy cho bài toán liệt kê tổ hợp, chỉnh hợp.

Bài 1. Liệt kê tổ hợp N số nguyên đầu tiên

Cho tập hợp gồm N số nguyên đầu tiên $\{1, 2, \dots, N\}$.

Yêu cầu: Hãy liệt kê tất cả tổ hợp chập K từ N phần tử của tập trên.

Dữ liệu: Nhập vào từ tập tin văn bản **TOHOP1.INP** gồm hai số N và K.

Kết quả: Xuất ra tập tin văn bản **TOHOP1.OUT** gồm:

- Dòng đầu chứa số M, với M là số lượng tổ hợp.
- M dòng tiếp, mỗi dòng chứa một tổ hợp giá trị tăng dần. Các tổ hợp được liệt kê theo thứ tự từ điển.

Ví dụ:

TOHOP1.INP
3 2

TOHOP1.OUT
3
1 2
1 3
2 3

Bài 2. Liệt kê chỉnh hợp N số nguyên đầu tiên

Cho tập hợp gồm N số nguyên đầu tiên $\{1, 2, \dots, N\}$.

Yêu cầu: Hãy liệt kê tất cả chỉnh hợp chập K từ N phần tử của tập trên.

Dữ liệu: Nhập vào từ tập tin văn bản **CHINHHOP1.INP** gồm hai số N và K.

Kết quả: Xuất ra tập tin văn bản **CHINHHOP1.OUT** gồm:

- Dòng đầu chứa số M, với M là số lượng chỉnh hợp.
- M dòng tiếp, mỗi dòng chứa một chỉnh hợp. Các chỉnh hợp được liệt kê theo thứ tự từ điển.

Ví dụ:

CHINHHOP1.INP
3 2

CHINHHOP1.OUT
6
1 2
1 3
2 1
2 3
3 1
3 2

Bài 3. Tổ hợp của tập hợp

Cho tập hợp A có N phần tử đôi một khác nhau. Mỗi phần tử là một ký tự từ $A \rightarrow Z$.

Yêu cầu: Hãy liệt kê các tổ hợp chập K từ N phần tử trong tập hợp đã cho.

Dữ liệu: Vào từ tập tin văn bản **TOHOP2.INP**, gồm:

- Dòng đầu tiên chứa N K với N là số lượng phần tử tập hợp A, K là số phần tử cần lấy tổ hợp.
- N dòng tiếp theo, mỗi dòng chứa 1 ký tự từ $A \rightarrow Z$ (các ký tự đôi một khác nhau).

Kết quả: Xuất ra tập tin văn bản **TOHOP2.OUT**, gồm:

- Dòng đầu chứa M – số lượng tổ hợp.

- M dòng tiếp theo, mỗi dòng chứa một tổ hợp.

Ví dụ:

TOHOP2.INP
3 2
F
B
A

TOHOP2.OUT
3
F B
F A
B A

Bài 4. Chỉnh hợp của tập hợp

Cho tập hợp A có N phần tử đôi một khác nhau. Mỗi phần tử là một ký tự từ A $\rightarrow$ Z.

Yêu cầu: Hãy liệt kê các chỉnh hợp chập K từ N phần tử của tập ký tự.

Dữ liệu: Vào từ tập tin văn bản **CHINHHOP2.INP**, gồm:

- Dòng đầu chứa N K – N là số lượng phần tử của tập hợp A, K là số phần tử cần lấy chỉnh hợp.
- N dòng tiếp theo, mỗi dòng chứa 1 ký tự từ A $\rightarrow$ Z (các ký tự đôi một khác nhau).

Kết quả: tập tin **CHINHHOP2.OUT**

- Dòng đầu ghi số nguyên M – số lượng chỉnh hợp.
- M dòng tiếp theo, mỗi dòng chứa một chỉnh hợp.

Ví dụ:

CHINHHOP2.INP
3 2
F
B
A

CHINHHOP2.OUT
6
F B
F A
B A
B F
A B
A F

Bài 5. Giá trị biểu thức

Cho hai số nguyên M và N. Hãy điền các phép toán +, -, *, %, / (chia nguyên) sao cho giá trị của biểu thức nhận được bằng đúng N:

$((((M ? M) ? M) ? M) ? M) ? M$

Dữ liệu: Nhập từ tập tin văn bản **BIEUTHUC.INP** chứa hai số M N.

Kết quả: Xuất ra tập tin văn bản **BIEUTHUC.OUT** với 7 toán tử trong bốn phép toán +, -, *, %, / (chia nguyên) (nằm trên đầu dòng) có giá trị bằng đúng N nếu có đáp án. Nếu không có, ghi số -1.

Ví dụ:

BIEUTHUC.INP
2 0
/* (((2 + 2) * 2) / 2) % 2 + 2 - */

BIEUTHUC.OUT
+
-
*
/
%
+
-

---o0o---
(Hết)

HƯỚNG DẪN THỰC HÀNH CƠ SỞ LẬP TRÌNH 2

TỔ HỢP, CHỈNH HỢP VÀ ỨNG DỤNG

---o0o---

Bài 1. TỔ HỢP

Khung chương trình cài đặt

```
#include <iostream>
#include <fstream>

using namespace std;

#define FI "TOHOP.INP"
#define FO "TOHOP.OUT"

/* Khai báo luồng nhập / xuất */
ifstream finp(FI);
ofstream fout(FO);

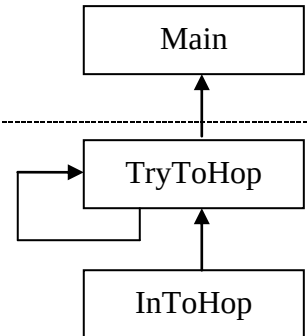
/* Biểu diễn cấu trúc dữ liệu */
#define MAX 100

int tohop[MAX+1]; // biểu diễn một tổ hợp tối đa 100 phần tử
int n, k;          // tổ hợp chập k từ n phần tử

/* Nguyên mẫu hàm */
/*
In tổ hợp n số nguyên đầu tiên ra màn hình
Input: tohop[]={1,3,5}, n = 5, k = 3
Output: xuất "1 3 5 xuống dòng"
*/
void InToHop();

/*
Sinh tổ hợp chập k từ n phần tử (đệ quy)
Input: tohop[], n, k, t (trạng thái đang xét)
Output: xuất tất cả các tổ hợp
*/
void TryToHop(int t);

int main(int argc, char *argv[])
{
    /* Kiểm tra luồng nhập/xuất mở thành công? */
    if(!finp.is_open() || !fout.is_open())
    {
        cout << "Luồng nhập/xuất mở không thành công?" << endl;
        return;
    } // if
    /* Xử lý luồng nhập/xuất */
    finp >> n >> k;
```



```

memset(tohop,-1,sizeof(tohop)); // điền giá trị -1 cho tohop[]
TryToHop(1);

/* Đóng luồng nhập/xuất */
finp.close();
fout.close();
} // main

/* Cài đặt các hàm */

```

Giải thích

Biểu diễn cấu trúc dữ liệu liệt kê tổ hợp:

```
/* Biểu diễn cấu trúc dữ liệu */
```

```
#define MAX 100
```

```
int tohop[MAX+1]; // biểu diễn một tổ hợp tối đa 100 phần tử
```

```
int n, k; // tổ hợp chập k từ n phần tử
```

tohop[i:1...k] biểu diễn tổ hợp chập k từ n phần tử số nguyên {1,2,...,n}. Ví dụ: n = 5, k = 3, tohop[1...3] = {1,4,5}.

```

/*
In tổ hợp n số nguyên đầu tiên ra màn hình
Input: tohop[]={1,3,5}, n = 5, k = 3
Output: xuất "1 3 5 xuống dòng"
*/
void InToHop();

```

Mã giả cài đặt:

```
InToHop()
```

```

1. for i ← 1 to k do
2.     In tohop[i] và khoảng trắng
3. In dấu xuống dòng
end

```

```

/*
Sinh tổ hợp chập k từ n phần tử (đệ quy)
Input: tohop[], n, k, t (trạng thái đang xét)
Output: xuất tất cả các tổ hợp
*/
void TryToHop(int t);

```

Liệt kê tổ hợp với phương pháp quay lui (dùng đệ quy):

Liệt kê các tổ hợp C_n^k với N=5 và K=3, ta có các tổ hợp sau:

1. {1, 2, 3} 4. {1, 3, 4} 6. {1, 4, 5} 7. {2, 3, 4} 9. {2, 4, 5} 10. {3, 4, 5}
2. {1, 2, 4} 5. {1, 3, 5} 8. {2, 3, 5}
3. {1, 2, 5}

Nhận xét:

- Tổ hợp đầu tiên $\{0, 1, 2\}$ có giá trị tăng dần từ $1 \dots k$ ($k = 3$).
- Liệt kê theo thứ tự tăng dần để tránh trùng.
- Tổ hợp cuối cùng là ba phần tử cuối của tập n số nguyên đầu tiên.

Do đó:

- Xét tập $\{1, 2, 3, \dots, n\}$ và liệt kê tổ hợp chập k từ n phần tử.
- Khi liệt kê một tổ hợp $\{a_1, a_2, \dots, a_k\}$, điều kiện $a_1 < a_2 < \dots < a_k$.
- Ta có:
 - o Trạng thái bắt đầu: $\{1, 2, 3, \dots, k-1, k\}$
 - o Trạng thái kết thúc: $\{n-k+1, n-k+2, n-k+3, \dots, n-1, n\}$
- Suy ra:
 - o Vị trí 1: $1 \leq a_1 \leq n-k+1$
 - o Vị trí 2: $2 \leq a_2 \leq n-k+2$
 - o ...
 - o Vị trí i : $i \leq a_i \leq n-k+i$
 - o ...
 - o Vị trí k : $k \leq a_k \leq n$

Xây dựng phương án quay lui (dùng đệ quy)

Để chọn một tổ hợp C_n^k , ta chia làm k giai đoạn (trạng thái) $V = \{v_1, v_2, \dots, v_i, \dots, v_k\}$. Mỗi giai đoạn có cách chọn lựa từ i đến $n-k+i$, tức là $v_i \in \{i, i+1, \dots, n-k+i\}$ sao cho thỏa $v_i > v_{i-1}$ (miền xác định S_i).

$v = \{ \begin{array}{|c|c|c|c|c|c|} \hline 1 \dots n-k+1 & 2 \dots n-k+2 & \dots & i \dots n-k+i & \dots & k \dots n \\ \hline 1 & 2 & \dots & i & \dots & k \\ \hline \end{array} \}$

V nằm trong không gian tổ hợp khi chọn đủ k phần tử lần lượt từ vị trí 1, 2, ..., k .

Mã giả cài đặt:

```

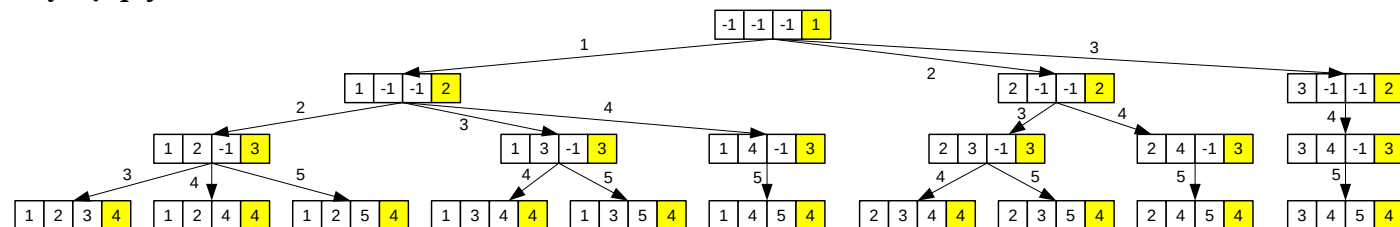
/*
Khởi tạo ban đầu:
    + tohop[] ← -1
    + TryToHop(1)
*/
TryToHop(t)
1. if t = k+1 then
2.     InToHop()
3.     return
4. min ← 1 khi t = 1 (vị trí đầu), tohop[t-1]+1 khi t>1
5. for i ← min to n - k + t do // chọn v_t có miền trị t ... n - k + t sao cho v_t > v_{t-1}
6.     tohop[t] ← i           // khởi tạo trạng thái trước khi tiến vào
7.     TryToHop(t+1)         // xét vị trí tiếp theo v_{t+1}
8.     tohop[t] ← -1         // phục hồi trạng thái
end
    
```

Lập bảng chạy tay, vẽ cây đệ quy, xác định thứ tự gọi hàm: với $n = 5, k = 3, \{1, 2, 3, 4, 5\}$

Bước	t	min	n - k + t	i	tohop[]			TryToHop	Ghi chú
					1	2	3		
0	1	---	---	---	-1	-1	-1	T ₁	Gọi T ₁
1	1	1	3	1	1	-1	-1	T ₁ T ₂	Gán giá trị 1 vào vị trí 1, vào T ₂
2	2	2	4	2	1	2	-1	T ₁ T ₂ T ₃	Gán giá trị 2 vào vị trí 2, vào T ₃
3	3	3	5	3	1	2	3	T ₁ T ₂ T ₃ T ₄	Gán giá trị 3 vào vị trí 3, vào T ₄
4	4	---	---	---	1	2	3	T ₁ T ₂ T ₃ T ₄	In ra màn hình 1 2 3, quay lui về T ₃
5	3	3	5	3	1	2	-1	T ₁ T ₂ T ₃	Phục hồi giá trị tại vị trí 3
6	3	3	5	4	1	2	4	T ₁ T ₂ T ₃ T ₄	Gán giá trị 4 vào vị trí 3, vào T ₄
7	4	---	---	---	1	2	4	T ₁ T ₂ T ₃ T ₄	In ra màn hình 1 2 4, quay lui về T ₃
8	3	3	5	4	1	2	-1	T ₁ T ₂ T ₃	Phục hồi giá trị tại vị trí 3

9	3	3	5	5	1	2	5	$T_1T_2T_3T_4$	Gán giá trị 5 vào vị trí 3, vào T_4
10	4	---	---	---	1	2	5	$T_1T_2T_3T_4$	In ra màn hình 1 2 5, quay lui về T_3
11	3	3	5	5	1	2	-1	$T_1T_2T_3$	Phục hồi giá trị tại vị trí 3, quay lui về T_2
12	2	2	4	2	1	-1	-1	T_1T_2	Phục hồi giá trị tại vị trí 2
13	2	2	4	3	1	3	-1	$T_1T_2T_3$	Gán giá trị 3 vào vị trí 2, vào T_3
14	3	4	5	4	1	3	4	$T_1T_2T_3T_4$	Gán giá trị 4 vào vị trí 3, vào T_4
...	...	...	...	...	...	...	...	...	...

Cây đệ quy



Thứ tự gọi hàm: T_1 gọi T_2 gọi T_3 gọi T_4 quay về T_3 ...

Cài đặt không đệ quy:

```
/*
Sinh tổ hợp chập k từ n phần tử (không đệ quy)
Input: n, k
Output: xuất tất cả các tổ hợp
*/
void TryToHop(int t);
```

Mã giả cài đặt:

```
TryToHop(t)
1. tohop[1...k] ← -1
2. t ← 1;          // bắt đầu tại vị trí 1
3. tohop[t] ← -1 // giá trị ban đầu ở vị trí 0
4. while t > 0 do  // vị trí vẫn chưa duyệt hết
5.     if t = n+1 then // sinh đủ trạng thái tổ hợp
6.         InToHop()
7.         t--          // lùi một bước
8.         continue     // lặp lại vòng lặp mới
9.     if tohop[t]+1 ≤ n - k + t then // vẫn còn tiến được
10.        tohop[t]++
11.        t++          // tiến vào trạng thái tiếp
12.        tohop[t] = tohop[t-1] // giá trị ban đầu cho trạng thái tiếp
13.    else
14.        tohop[t] ← -1 // phục hồi trạng thái
15.        t--          // lùi một bước
end
```

Lập bảng chạy tay chương trình quay lui không đệ quy.

---o0o---
(Hết)

THỰC HÀNH CƠ SỞ LẬP TRÌNH 2

BÀI TOÁN XẾP HẬU VÀ MÃ ĐI TUẦN

---o0o---

YÊU CẦU

- Ứng dụng phương pháp quay lui.
- Cài đặt chương trình đệ quy (khử đệ quy) và không đệ quy.

Bài 1. XẾP HẬU

Xét bàn cờ tổng quát kích thước $N \times N$. Một quân hậu trên bàn cờ có thể ăn được các quân khác nằm tại các ô cùng hàng, cùng cột hoặc cùng đường chéo. Hãy liệt kê tất cả các cách xếp N quân hậu trên bàn cờ sao cho không có quân hậu nào ăn nhau. ($3 \leq N \leq 100$). *(Làm hai cách: hoán vị và quay lui)*

Dữ liệu vào: tập tin **XEPHAU.INP** chứa số nguyên N với N là kích thước cạnh của bàn cờ $N \times N$.

Dữ liệu ra: tập tin **XEPHAU.OUT**

- Dòng đầu tiên ghi số 0 hoặc 1 với 0 là không có cách xếp và 1 là có cách xếp.
- Nếu có, N dòng tiếp theo, mỗi dòng có N số cách nhau bởi một khoảng trắng biểu diễn bàn cờ và cách xếp hậu. Nếu giá trị là số 0 nghĩa là không có hậu. Ngược lại là 1 nghĩa là có hậu đặt ở vị trí đó.

Ví dụ:

XEPHAU.INP	XEPHAU.OUT
5	1 1 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 1 0
2	0

Câu hỏi:

1. Biểu diễn dữ liệu cho bài toán xếp hậu.
2. Ứng dụng phương pháp quay lui:
 - a. Nêu ý tưởng giải quyết bài toán dựa trên phương pháp quay lui.
 - b. Nêu thuật toán giải quyết bài toán.
 - c. Lập bảng chạy tay và vẽ cây đệ quy để minh họa thuật toán.
3. Lập trình giải quyết bằng ngôn ngữ C/C++ (đệ quy - khử đệ quy, không đệ quy).

Bài 2. MÃ ĐI TUẦN

Trong bàn cờ tổng quát kích thước $N \times N$, con mã được đặt tại vị trí x_0, y_0 . Hãy chỉ ra lộ trình con mã đi qua tất cả các ô mà mỗi ô chỉ đi qua đúng một lần và trở lại vị trí ban đầu. ($2 \leq N \leq 10$).

Dữ liệu vào: tập tin **MADITUAN.INP**

- Dòng đầu chứa 3 số nguyên: N x_0 y_0 , ứng với N là kích thước một cạnh của bàn cờ và (x_0, y_0) là tọa độ ô đầu tiên của quân mã trên bàn cờ.

Dữ liệu ra: tập tin **MADITUAN.OUT**

- Dòng đầu tiên ghi số 0 hoặc 1 với 0 là không có lộ trình đi và 1 là có lộ trình đi.

- Nếu có, N dòng tiếp theo, mỗi dòng có N số cách nhau bởi một khoảng trắng biểu diễn bàn cờ và lộ trình đi của con mã. Nếu giá trị ô đó là k, nghĩa là con mã đi qua ô đó của bàn cờ bằng nước đi thứ k. Ô (x0,y0) coi như là nước đi thứ 1.

Ví dụ:

MADITUAN.INP	MADITUAN.OUT
6 0 0	1 22 27 30 3 20 26 31 2 21 8 29 23 36 25 28 19 4 32 13 34 7 16 9 35 24 11 14 5 18 12 33 6 17 10 15
3 0 0	0

Câu hỏi: hãy bổ sung điều kiện nhánh cận để cải tiến tốc độ cho thuật toán phương pháp quay lui bài toán mã đi tuần.

---o0o---
(Hết)

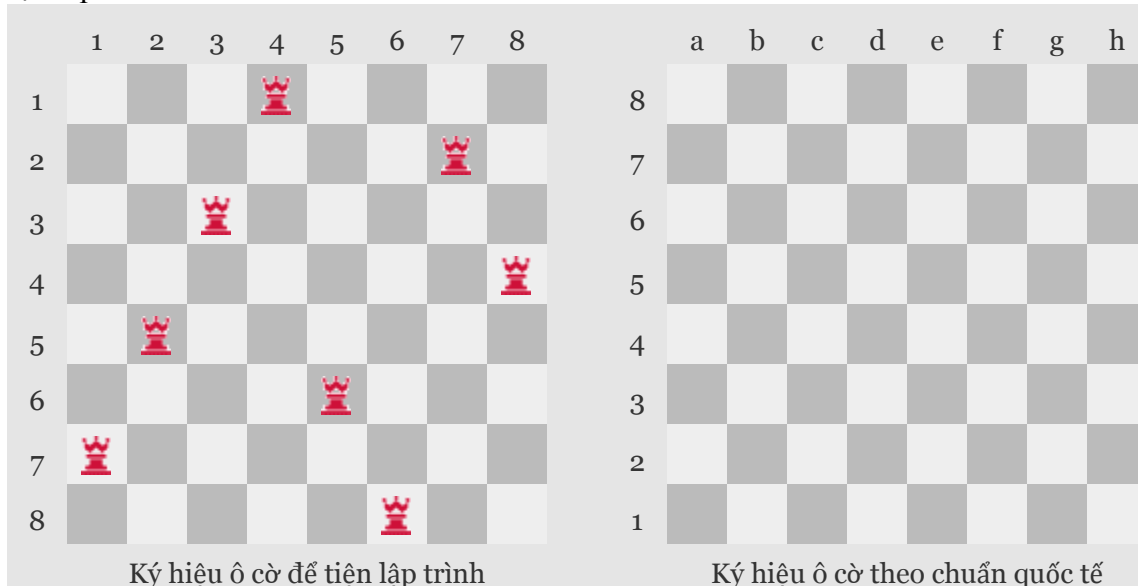
BÀI 1. BÀI TOÁN XẾP HẬU

XEPHAU.*

1. Mô tả bài toán:

Bài toán tìm tất cả cách đặt n con hậu trên bàn cờ vua có kích thước $n \times n$ ($n \geq 4$) sao cho không có con hậu nào khống chế lẫn nhau. Biết một con hậu có thể khống chế tất cả quân cờ trên hàng ngang, cột dọc và hai đường chéo đi qua vị trí của nó.

Ví dụ một đáp án của bài toán:



Bàn cờ vua 8x8 (ký hiệu các ô cờ và ví dụ đặt 8 quân hậu)

2. Biểu diễn kết quả:

```
/* Biểu diễn kết quả */
#define MAX 12

int hau[MAX+1]; // biểu diễn vị trí của n con hậu
int n;           // độ lớn của bàn cờ
```

Do yêu cầu của bài toán là tìm các vị trí để đặt n quân hậu nên thay vì sử dụng mảng 2 chiều, ta chỉ cần sử dụng mảng một chiều `hau[1..n]` trong đó `hau[i]` thuộc đoạn `[1..n]` là vị trí cột của quân hậu ở dòng thứ i .

Ví dụ kết quả biểu diễn mảng `hau` ở hình trên:

i	1	2	3	4	5	6	7	8
hau[i]	4	7	3	8	2	5	1	6

Nghĩa là, quân hậu dòng 1 sẽ ở cột 3, quân hậu dòng 2 sẽ ở cột 7, ...

InKetQua(`hau[1..n]`, n): In kết quả ra màn hình với ô 1 là có hậu và ô 0 không có hậu

```
for d=1,2,..., n do
  for c=1,2,...,n do
    if hau[d]==c then
      In 1
    else
      In 0
  Xuống dòng
```

3. Ý tưởng đệ quy quay lui:

Cách 1. Hoán vị

- Mỗi lời giải của bài toán tám hậu chính là dãy giá trị $\{hau[1], hau[2], \dots, hau[n]\}$ có miền giá trị trong đoạn $[1, n]$ và đôi một khác nhau. Đây chính là dãy giá trị hoán vị của dãy số $\{1, 2, \dots, n\}$. Khi đó, tập n con hậu có tính chất không bị khống chế về dòng (do giá trị chỉ số không trùng) và không bị khống chế về cột (do cặp giá trị trong dãy đôi một khác nhau).
- Do đó, bài toán xếp hậu đưa về bài liệt kê hoán vị (tập không gian P) và kiểm tra hoán vị có thỏa mãn cách đặt con hậu không bị khống chế ở hai đường đi qua vị trí hậu (điều kiện Q).
- Khung đệ quy quay lui cho Cách 1:

```

/*
Cấu trúc dữ liệu:
+ dau[1..n] = 0 (chưa xét), 1 (đã xét)
Khởi tạo ban đầu:
+ hau[1..n] ← -1
+ day[1..n] ← 0
+ TryXepHau(1)
*/
TryXepHau(k) // xét hậu ở vị trí dòng k
  for  $v_k=1, 2, \dots, n$  do
    if dau[vk] == 0 then
      hau[k] ← vk
      dau[vk] ← 1
      if k==n then
        if LaDayHopLe(hau[1..n], n) then
          InKetQua(hau[1..n], n)
        else
          TryXepHau(k+1)
      hau[k] ← -1
      dau[vk] ← 0
End

```

Cách 2. Kiểm tra đủ bốn phương

- Gọi $\vec{v} = \{v_1, v_2, \dots, v_n\}$ là biểu diễn trạng thái lời giải với v_i là cách đặt con hậu i (ở dòng thứ i) tại cột v_i . Do đó, cách đặt giả thiết này sẽ không có cặp hậu khống chế lẫn nhau về dòng.
- Miền trị:

$$v_i \in D_i \triangleq \begin{cases} v_i = 1, 2, \dots, n \\ \text{Ô}(i, v_i) \text{ không bị khống chế bởi các con hậu } 1, 2, \dots, i-1 \end{cases}$$

- $\vec{v}$ là đáp án bài toán xếp hậu khi đặt đủ n con hậu, nghĩa là $|\vec{v}| = n$
- Khung đệ quy quay lui cho Cách 2:

```

/*
Cấu trúc dữ liệu:
+ CTDL biểu diễn Trạng thái không chế
Khởi tạo ban đầu:
+ hau[1..n] ← -1
+ Khởi tạo CTDL biểu diễn trạng thái không chế
+ TryXepHau(1)
*/
TryXepHau(k) // xét hậu ở vị trí dòng k
  for  $v_k=1, 2, \dots, n$  do
    if LaHopLe(k,  $v_k$ ) == true then
      DatHau(k,  $v_k$ )
      if k==n then
        InKetQua(hau[1..n], n)
      else

```

```
TryXepHau (k+1)
BoHau (k, vk)
```

End

4. Biểu diễn sự không chế của hậu trên bàn cờ:

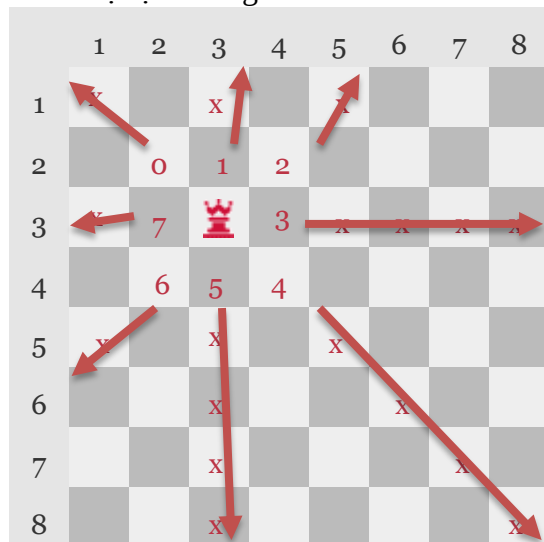
Cách 1. Xét ở mức ô

```
/* Cấu trúc dữ liệu */
// banco[d][c] = 0 / k, nghĩa là ô (d,c) chưa bị / đã bị không chế
int banco[MAX+1][MAX+1];

/* Khởi tạo ban đầu */
#include <cstring>
...
memset(banco, 0, sizeof(banco)); // giả sử tất cả các ô chưa bị không chế
```

DatHau(d, c): tăng các ô bị hậu không chế lên 1

Các ô bị hậu không chế:



Mảng `huong[8][2]` đánh dấu sự di chuyển của hậu theo 8 hướng với sự thay đổi về dòng, cột:

huong	0 (d)	1 (c)
0	-1	-1
1	-1	0
2	-1	1
3	0	1
4	1	1
5	1	0
6	1	-1
7	0	-1

Ví dụ hậu ở ô (d,c) và di chuyển theo hướng 1:
 $(dn, cn) = (d + \text{huong}[1][0], c + \text{huong}[1][1])$
 $\rightarrow (dn, cn) = (d - 1, c)$

```
int huong[8][2] = { {-1,-1}, {-1,0}, ..., {0,-1} };
banco[d][c] ← banco[d][c] + 1
for h=0,1,...,7 do
    dn ← d + huong[h][0]
    cn ← c + huong[h][1]
    while dn >= 0 và dn <= n và cn >= 0 và cn <= n do
        banco[dn][cn] ← banco[dn][cn] + 1
        dn ← d + huong[h][0]
        cn ← c + huong[h][1]
```

BoHau(d, c): giảm các ô bị hậu không chế xuống 1

```
int huong[8][2] = { {-1,-1}, {-1,0}, ..., {0,-1} };
banco[d][c] ← banco[d][c] - 1
for h=0,1,...,7 do
    dn ← d + huong[h][0]
    cn ← c + huong[h][1]
    while dn >= 1 và dn <= n và cn >= 1 và cn <= n do
        banco[dn][cn] ← banco[dn][cn] - 1
        dn ← dn + huong[h][0]
        cn ← cn + huong[h][1]
```

LaHopLe(d, c): kiểm tra ô (d,c) có hợp lệ ứng với trạng thái bàn cờ xác định

```
if banco[d][c] > 0 then
    return false
else
    return true
```

LaDayHopLe (hau[1..n], n) : kiểm tra dãy {hau[1], hau[2], ..., hau[n]} khi đặt lần lượt lên bàn cờ trống có hợp lệ

```
memset(banco, 0, sizeof(banco)); // #include <cstring>
DatHau(1, hau[1])
for d=2,...,n do
    if LaHopLe(d, hau[d]) == true then
        DatHau(d, hau[d])
    else
        return false
return true
```

Cách 2. Xét bốn phương: ngang, dọc, chéo chính, chéo phụ

- **Ý tưởng**: khi kiểm tra tính hợp lệ của hậu đặt ở 1 ô, ta chỉ cần kiểm tra ô đó có nằm trên các dòng (phương ngang), cột (phương dọc) và chéo chính, chéo phụ tương ứng có bị không chế.

	1	2	3	4	5	6	7	8
1	1	1	1	1	1	1	1	1
2	2	2	2	2	2	2	2	2
3	3	3	3	3	3	3	3	3
4	4	4	4	4	4	4	4	4
5	5	5	5	5	5	5	5	5
6	6	6	6	6	6	6	6	6
7	7	7	7	7	7	7	7	7
8	8	8	8	8	8	8	8	8

Ngang

	1	2	3	4	5	6	7	8
1	1	2	3	4	5	6	7	8
2	1	2	3	4	5	6	7	8
3	1	2	3	4	5	6	7	8
4	1	2	3	4	5	6	7	8
5	1	2	3	4	5	6	7	8
6	1	2	3	4	5	6	7	8
7	1	2	3	4	5	6	7	8
8	1	2	3	4	5	6	7	8

Dọc

	1	2	3	4	5	6	7	8
1	0	-1	-2	-3	-4	-5	-6	-7
2	1	0	-1	-2	-3	-4	-5	-6
3	2	1	0	-1	-2	-3	-4	-5
4	3	2	1	0	-1	-2	-3	-4
5	4	3	2	1	0	-1	-2	-3
6	5	4	3	2	1	0	-1	-2
7	6	5	4	3	2	1	0	-1
8	7	6	5	4	3	2	1	0

Chéo chính

	1	2	3	4	5	6	7	8
1	2	3	4	5	6	7	8	9
2	3	4	5	6	7	8	9	10
3	4	5	6	7	8	9	10	11
4	5	6	7	8	9	10	11	12
5	6	7	8	9	10	11	12	13
6	7	8	9	10	11	12	13	14
7	8	9	10	11	12	13	14	15
8	9	10	11	12	13	14	15	16

Chéo phụ

- **Xét phương ngang** (các dòng của bàn cờ):
 - o Bàn cờ có tất cả 8 phương ngang đánh số từ 1 đến 8
 - o Ô (d,c) chỉ nằm ở phương ngang thứ d
 - o Do cách đặt giả thiết hậu i ở dòng thứ i nên ta không cần xét đến phương ngang
- **Xét phương dọc** (các cột của bàn cờ):
 - o Bàn cờ có tất cả 8 phương dọc đánh số từ 1 đến 8
 - o Ô (d,c) chỉ nằm ở phương dọc thứ c
 - o Biểu diễn dữ liệu: $\text{cot}[1..N] = 0 / 1$ nghĩa là chưa không chế / đã không chế
- **Xét phương chéo chính:**
 - o Phương chéo chính có tính chất các ô cùng phương có hiệu của dòng và cột là hằng số. Do đó, ta có: $1 - n \leq d - c \leq n - 1$
 $\Rightarrow 1 - n + n \leq d - c + n \leq n - 1 + n$
 $\Rightarrow 1 \leq d - c + n \leq 2n - 1$
 - o Ô (d,c) chỉ nằm ở phương chéo chính thứ $d - c + n$
 - o Biểu diễn dữ liệu: $\text{cheo1}[1..2N-1] = 0 / 1$ nghĩa là chưa không chế / đã không chế
- **Xét phương chéo phụ:**
 - o Phương chéo phụ có tính chất các ô cùng dòng có tổng của dòng và cột là hằng số. Do đó, ta có: $2 \leq d + c \leq 2n$
 $\Rightarrow 2 - 1 \leq d + c - 1 \leq 2n - 1$
 $\Rightarrow 1 \leq d + c - 1 \leq 2n - 1$
 - o Ô (d,c) chỉ nằm ở phương chéo chính thứ $d + c - 1$
 - o Biểu diễn dữ liệu: $\text{cheo2}[1..2N-1] = 0 / 1$ nghĩa là chưa không chế / đã không chế

```

/* Cấu trúc dữ liệu */
int cot[MAX+1];
int cheo1[2*MAX];
int cheo2[2*MAX];

/* Khởi tạo ban đầu */
#include<cstring>
...
memset(cot, 0, sizeof(cot)); // giả sử tất cả các ô chưa bị không chế
memset(cheo1, 0, sizeof(cheo1)); // giả sử tất cả các ô chưa bị không chế
memset(cheo2, 0, sizeof(cheo2)); // giả sử tất cả các ô chưa bị không chế

```

DatHau(d, c): đánh dấu các phương đi ngang qua hậu ở (d,c) đã bị không chế

```

hau[d] ← c
cot[c] ← 1
cheo1[d - c + n] ← 1
cheo2[d + c - 1] ← 1

```

BoHau(d, c): đánh dấu các phương đi ngang qua hậu ở (d,c) chưa bị không chế

```

hau[d] ← -1
cot[c] ← 0
cheo1[d - c + n] ← 0
cheo2[d + c - 1] ← 0

```

LaHopLe(d, c): kiểm tra ô (d,c) có hợp lệ ứng với trạng thái bàn cờ xác định

```

if cot[c]==1 và cheo1[d - c + n]==1 và cheo2[d + c - 1]==1 then
  return false

```

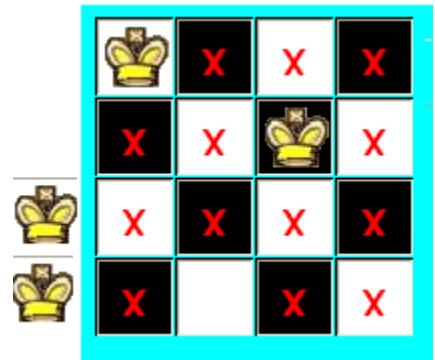
```
else
    return true
```

LaDayHopLe (hau[1..n], n) : kiểm tra dãy {hau[1],hau[2],...,hau[n]} khi đặt lần lượt lên bàn cờ trống có hợp lệ

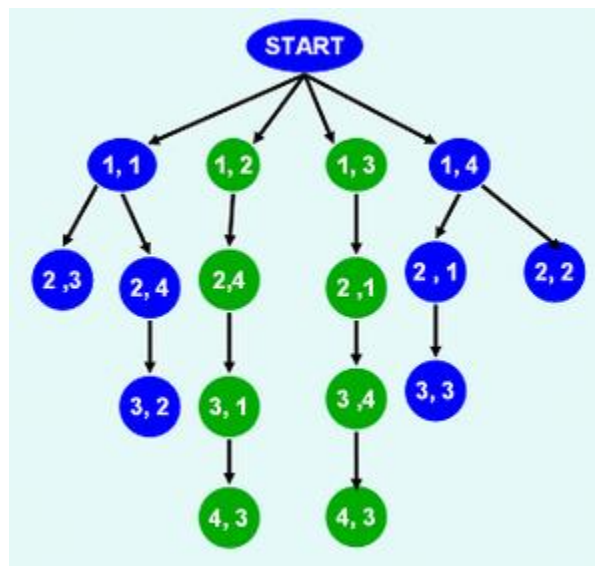
```
memset(cheo1,0, sizeof(cheo1)); // #include <cstring>
memset(cheo2,0, sizeof(cheo2));
memset(cot,0, sizeof(cot));
DatHau(1, hau[1])
for d=2,...,n do
    if LaHopLe(d, hau[d]) == true then
        DatHau(d, hau[d])
    else
        return false
return true
```

5. Chạy tay:

Ta minh họa quá trình tìm kiếm lời giải cho bài toán n hậu với n =4 trong hình bên. Ở trạng thái xuất phát, trên dòng 1 có 4 lựa chọn cho quân hậu: quân hậu thứ nhất có thể đứng ở các cột 1,2,3,4. Nếu lựa chọn ô (1,1), ở dòng thứ hai chỉ còn hai lựa chọn là cột 3 và cột 4. Nếu lựa chọn cột 3, trên dòng thứ 3 sẽ không còn ô nào không bị khống chế ô (3,1) và (3,3) khống chế bởi (1,1), ô (3,2) và (3,4) khống chế bởi (2,4). Ta loại bỏ phương án chọn ô (2,3) này và xét tiếp phương án chọn ô (2,4). Khi lựa chọn ô (2,4) ta cũng chỉ đặt thêm được một quân hậu ở dòng thứ ba. Dòng thứ tư lại không thể đặt bất kỳ quân hậu nào. Do đó ta lùi lại dòng thứ nhất, xét khả năng tiếp theo (1,2), ta lần lượt được dãy các ô (1,2), (2,4), (3,1), (4,3). Tiếp tục với ô (1,3), (1,4). Chỉ có hai đường đi từ gốc tới lá với độ dài 4 nên bài toán 4 hậu chỉ có 2 lời giải thể hiện trên cây bằng các đường đi màu xanh lục.



Cây đệ quy ứng với bàn cờ 4:



6. Thảo luận:

- Hãy đánh giá ưu điểm / nhược điểm bài toán xếp hậu dùng hoán vị hoặc dùng đệ quy quay lui khống chế bốn phương cùng với kiểm tra khống chế theo từng ô và theo phương.

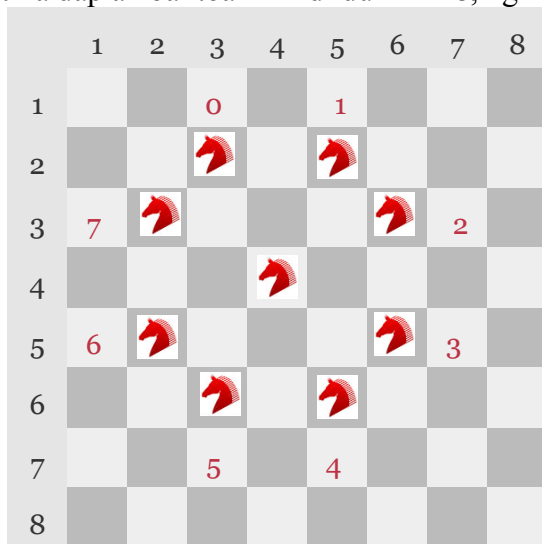
- Cài đặt bài toán xếp hậu theo tổ hợp bốn cách: (hoán vị, đệ quy quay lui theo phương) x (khống chế theo ô, khống chế theo phương).
- Mỗi hậu có n cách đi, hãy chọn tiêu chí đánh giá coi nên đi cách nào trước, cách nào sau (heuristic). Cài bài toán theo heuristic vừa đề xuất.

BÀI 2. MÃ ĐI TUẦN**MADITUAN.*****Ý tưởng đệ quy quay lui:**

- Gọi (d_0, c_0) là ô đầu tiên của con mã chuẩn bị đi. Con mã có 8 hướng đi đánh số từ 0 đến 7.
- Gọi $\vec{v} = \{v_1, v_2, \dots, v_m\}$ là biểu diễn trạng thái lời giải với v_i là hướng đi con mã ở nước thứ i .
- Miền trị:

$$v_i \in D_i \triangleq \begin{cases} v_i = \{0, 1, 2, 3, 4, 5, 6, 7\} \\ \text{Ô ở nước đi thứ } i \text{ hợp lệ và chưa được đi qua} \end{cases}$$

- $\vec{v}$ là đáp án bài toán khi đi đủ $n \times n$ ô, nghĩa là $|\vec{v}| = n \times n - 1$



huong	0 (d)	1 (c)
0	-2	-1
1	-2	1
2	-1	2
3	1	2
4	2	1
5	2	-1
6	1	-2
7	-1	-2

```

/*
Cấu trúc dữ liệu:
+ banco[1..MAX][1..MAX], n, (d0, c0)
+ dau[1..MAX][1..MAX]
+ huong[8][2] = { {-2,-1}, {-2,1}, ..., {-1,-2} }
+ di, ci
Khởi tạo ban đầu:
+ banco[1..MAX][1..MAX] ← 0
+ banco[d0][c0] ← 1
+ (di,ci) ← (d0,c0)
+ TryMaDiTuan(1)
*/
TryMaDiTuan(k) // xét con mã ở nước đi thứ k
for v_k=0,1,...,7 do
- (di,ci) ← (di + huong[v_k][0], ci + huong[v_k][1])
- if (di>=1 và di<=n và ci>=1 và ci<=n) và banco[di][ci]==0 then
+ banco[di][ci] ← k + 1
+ if k==n then
InKetQua(banco[1..n][1..n], n)
else
TryMaDiTuan(k+1)

```



```
+ banco[di][ci] ← 0  
- (di, ci) ← (di - huong[vk][0], ci - huong[vk][1])
```

End

- Mỗi vị trí của mã có 8 cách đi, hãy chọn tiêu chí đánh giá coi nên đi cách nào trước, cách nào sau (heuristic). Cài bài toán theo heuristic vừa đề xuất.

--- Hết ---